EXPLAINABLE - DRIFT-AWARE - COST-ACCOUNTED

TabML-Guard Report — Room Occupancy Detection

data/occupancy.csv · 20,559 rows × 5 features · classification

RECOMMENDED MODEL

random_forest_bayes

ROC_AUC (HELD-OUT)

0.9976

AT A GLANCE

- Tuning: marginal gain
- Inputs drifted, performance held

Headline finding

random_forest_bayes is the recommended model, scoring 0.9976 roc_auc on a strictly held-out, time-ordered test set.

Recommendation

Hyperparameter tuning moved the score by less than 1% over defaults, so the default configuration is a defensible production choice. At an equal budget Bayesian search edged out random search, but cost 3.5x the wall-clock. The input distribution shifted materially (PSI above 0.25) while accuracy held up - monitor, but no retrain is urgent yet.

This report was produced automatically by the TabML-Guard pipeline. Every number below comes from a single run on a strictly held-out, time-ordered test split - no test data was used for model selection or tuning.

CONTENTS

Table of contents

Dataset, profile and configuration	3
Regularized linear baselines	3
Ensemble and boosting models	3
Hyperparameter optimization: benefit vs cost	4
Explainability and prediction reliability	4
Concept drift and statistical significance	6
Appendix A - Scope and limitations	8
Appendix B - Glossary	8

STAGE 0

Dataset, profile and configuration

The pipeline begins by profiling the raw table and deciding, from the target column alone, whether this is a classification or a regression problem. When a time column is present the split is chronological - the model trains on the past and is scored on the future.

Rows	20,559
Features	5
Target column	Occupancy
Detected task	classification
Missing values	0
Time column	date
Time span	2015-02-02 14:19:00 -> 2015-02-18 09:19:00
Class balance	0=0.769 1=0.231

Configuration used

This is the entire dataset-specific input to the run. Everything else is generic pipeline code.

```
dataset.source        csv
dataset.name          -
dataset.path          data/occupancy.csv
dataset.target        Occupancy
dataset.time_col      date
dataset.drop_cols     []
split                 train=0.6  test=0.4
models.boosters       ['xgboost', 'lightgbm', 'catboost', 'random_forest']
models.stacking       True
tuning.budget_trials  40
tuning.cv_splits      3
drift.n_chunks        10
drift.retrain         sliding
random_state          42
```

STAGE 1

Regularized linear baselines

Before reaching for an ensemble, the pipeline establishes what a well-regularized linear model can do. This is the honesty check: if a linear model is competitive, the extra complexity of boosting has to justify itself.

model	roc_auc	accuracy	f1	precision	recall	fit
logreg_none	**0.9967**	**0.9931**	**0.9833**	**0.9704**	**0.9964**	**0.0s**
logreg_elasticnet	0.9967	0.9931	0.9833	0.9704	0.9964	0.1s
logreg_l2	0.9967	0.9931	0.9833	0.9704	0.9964	0.0s
logreg_l1	0.9967	0.9931	0.9833	0.9704	0.9964	0.0s

the 4 regularizers span only 0.0000 roc_auc (best 'logreg_none' 0.9967) — with abundant rows and few features, penalties barely move the fit at the default strength; L1 path: at C=0.001 only 2/5 features survive (sparsity), rising to 5/5 as C grows

STAGE 2

Ensemble and boosting models

Gradient-boosted trees plus a stacking meta-model, all scored through the same evaluation path as the baselines so the leaderboard rows are directly comparable.

model	roc_auc	accuracy	f1	precision	recall	fit
random_forest	**0.9970**	**0.9846**	**0.9635**	**0.9321**	**0.9970**	**0.5s**
stacking	0.9969	0.9796	0.9523	0.9100	0.9988	9.2s
logreg_none	0.9967	0.9931	0.9833	0.9704	0.9964	0.0s
logreg_elasticnet	0.9967	0.9931	0.9833	0.9704	0.9964	0.1s
logreg_l2	0.9967	0.9931	0.9833	0.9704	0.9964	0.0s
logreg_l1	0.9967	0.9931	0.9833	0.9704	0.9964	0.0s
catboost	0.9965	0.9826	0.9591	0.9225	0.9988	1.6s
xgboost	0.9934	0.9672	0.9255	0.8623	0.9988	0.3s
lightgbm	0.9890	0.9630	0.9169	0.8475	0.9988	0.5s

Best ensemble: **random_forest**. The strongest single booster (**random_forest**) is the one carried into the tuning comparison in Stage 3.

STAGE 3

Hyperparameter optimization: benefit vs cost

Four strategies compete on **random_forest**: library defaults, an exhaustive coarse grid, random search, and Bayesian optimization (Optuna TPE). Random and Bayesian are given an **equal budget** of 40 trials, which makes them a fair head-to-head. Crucially, the table reports not just the score but what each strategy *cost*.

strategy	test roc_auc	wall clock	model fits	cv score
default	0.9968	1.1s	1	-
grid	0.9974	5.2s	54	-
random	0.9974	7.9s	120	-
bayes	**0.9976**	**27.8s**	**120**	**0.9856**

The *cv score* column is the internal cross-validated score each search optimized (scikit-learn's convention is higher-is-better, so error metrics appear negated). The *test* column is the winning configuration refit once and scored on the untouched held-out period - that is the number to trust.

Is tuning worth it?

best tuner 'bayes' reached roc_auc=0.9976 vs default 0.9968 (+0.0008); at equal budget, Bayesian search won (roc_auc: bayes 0.9976 @ 27.78s vs random 0.9974 @ 7.94s)

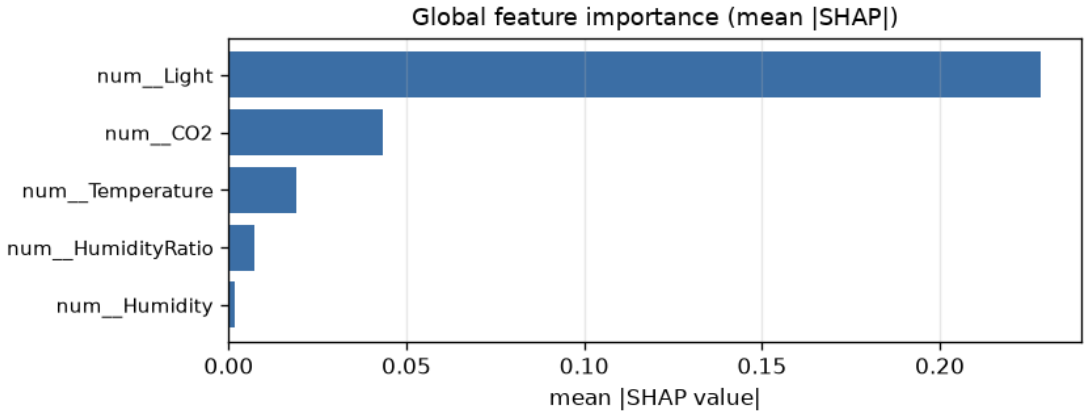
STAGE 4

Explainability and prediction reliability

SHAP attributions for **random_forest_bayes**, using the explainer matched to the model family (tree, linear, or model-agnostic fallback). Global importance answers *what drives this model overall*; the local example shows how a

single prediction was reached.

feature	mean |SHAP|
num__Light	0.22848
num__CO2	0.04325
num__Temperature	0.01896
num__HumidityRatio	0.00741
num__Humidity	0.00162



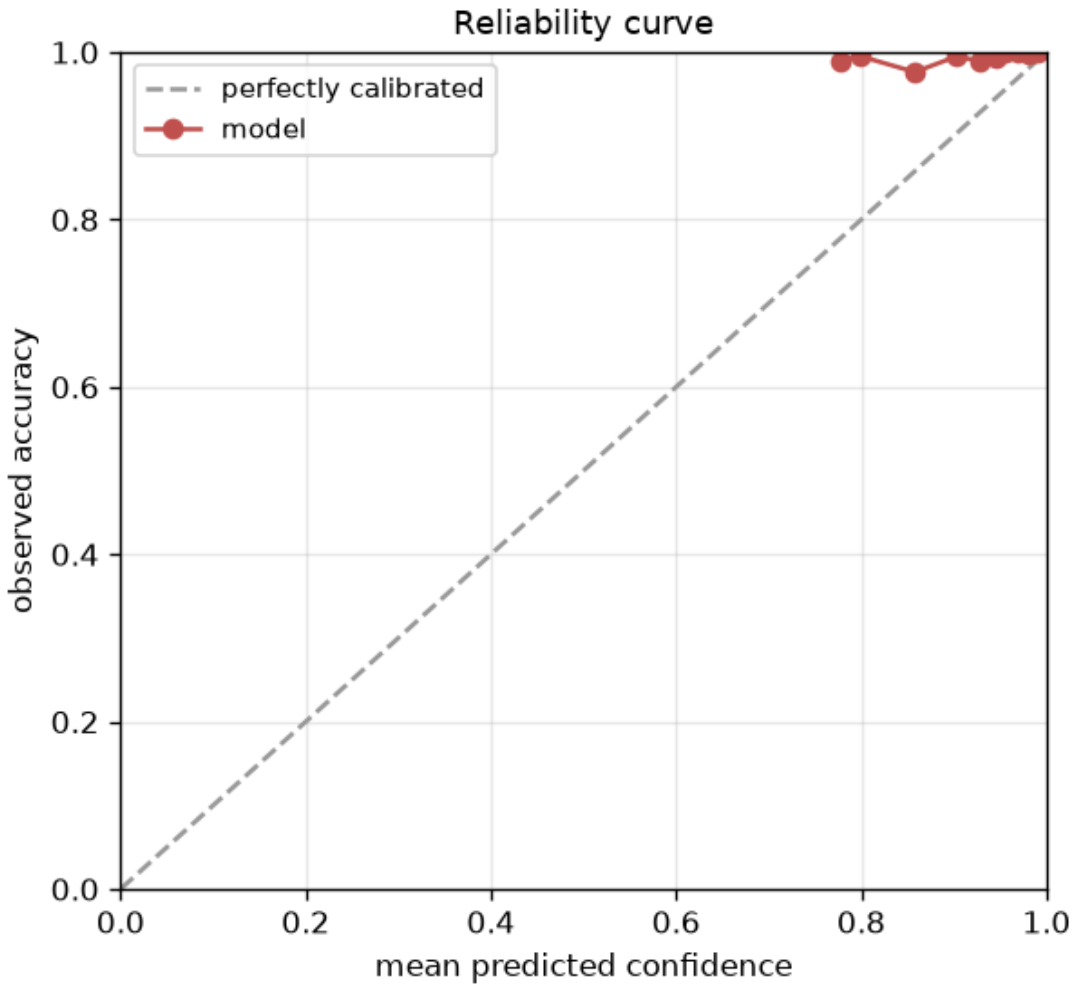
Global feature importance (mean |SHAP|)

Confidence-based triage

A model that knows when it is unsure is far more useful in production than one that is uniformly confident. Binning the held-out predictions by confidence shows whether confidence actually tracks correctness - and if it does, the least-confident slice can be routed to a human.

confidence band	n	mean confidence	accuracy
least confident 1/10	756	0.778	98.9%
2/10	887	0.798	99.6%
3/10	803	0.858	97.6%
4/10	837	0.903	99.5%
5/10	828	0.928	98.9%
6/10	823	0.946	99.3%
7/10	823	0.955	100.0%
8/10	817	0.970	100.0%
9/10	826	0.980	99.8%
most confident 10/10	824	0.990	100.0%

Flagged for review	20.1%
Of all errors, caught	23.1%
Accuracy on auto-approved	99.4%
Accuracy on flagged	99.3%
Confidence cut-off	0.809



Reliability curve: confidence vs observed accuracy

STAGE 5

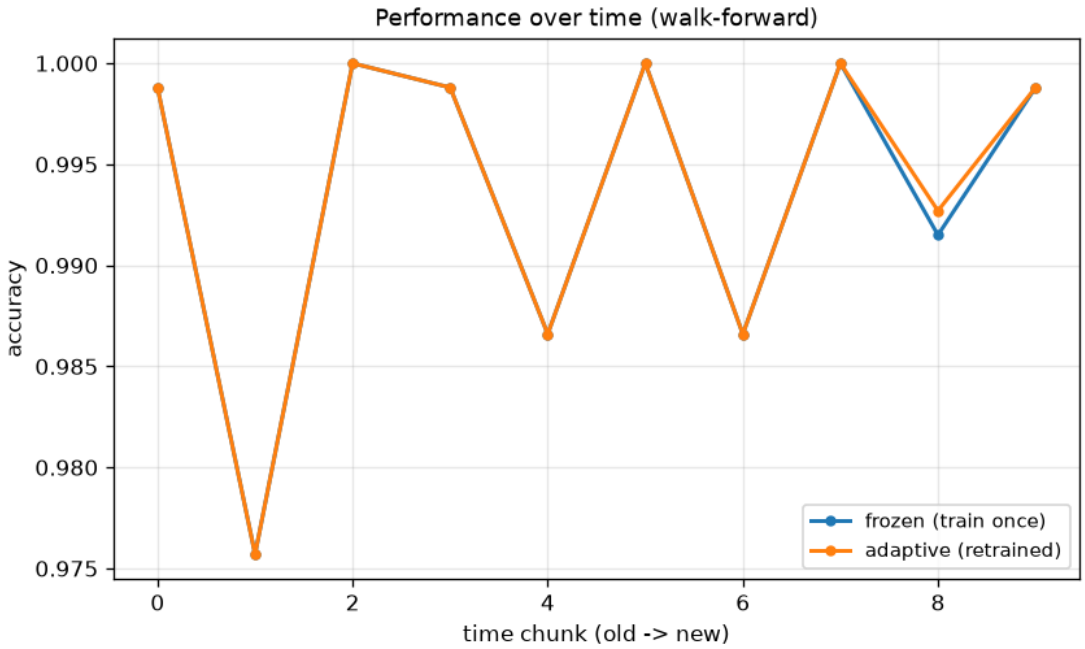
Concept drift and statistical significance

The held-out period is replayed in 10 chronological chunks. A **frozen** model (trained once, never updated) is compared against an **adaptive** model that predicts each chunk *before* learning from it - so there is no look-ahead. Retraining mode: **sliding**.

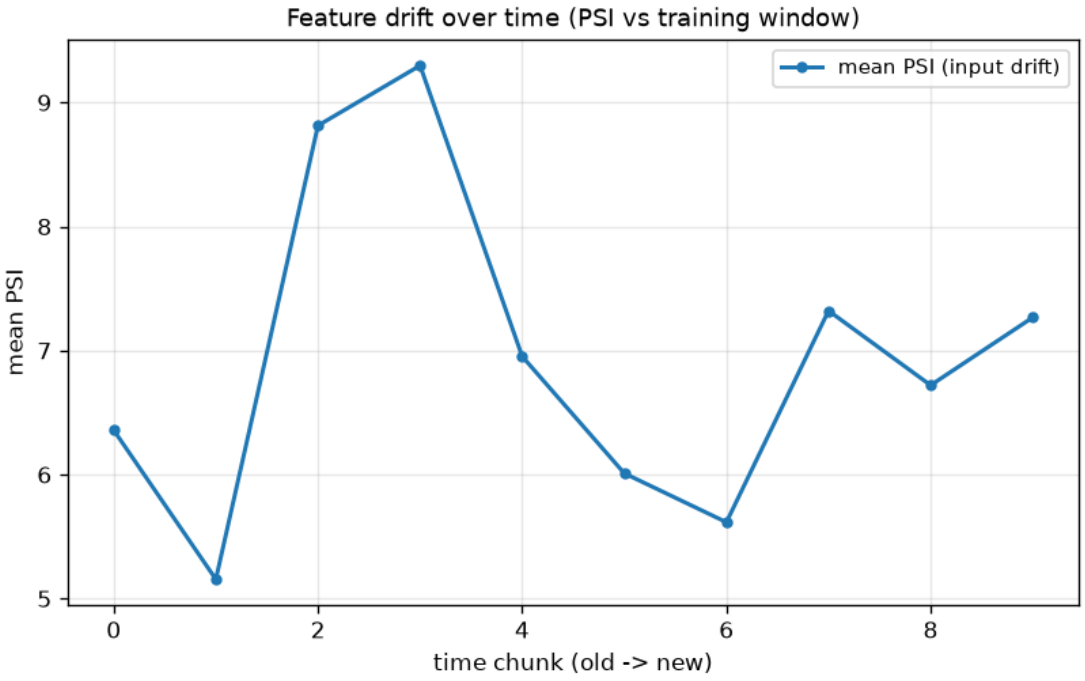
chunk	rows	frozen	adaptive	mean PSI
0	823	0.9988	0.9988	6.358
1	823	0.9757	0.9757	5.156
2	823	1.0000	1.0000	8.814
3	823	0.9988	0.9988	9.298
4	822	0.9866	0.9866	6.953
5	822	1.0000	1.0000	6.010
6	822	0.9866	0.9866	5.616
7	822	1.0000	1.0000	7.321

chunk	rows	frozen	adaptive	mean PSI
8	822	0.9915	0.9927	6.720
9	822	0.9988	0.9988	7.267

Chunk 1 is a built-in consistency check: both models are identical there by construction, since neither has seen the chunk yet.



Frozen vs adaptive performance across the held-out period



Input drift (PSI) against the training window

Mean score, frozen model	0.9937
Mean score, adaptive model	0.9938
First third -> last third	- -> -

Performance worsened	no
Worst-drifting feature	-
Peak mean PSI	9.298

Drift verdict

Frozen-model accuracy held up across time (0.999 -> 0.999; drift magnitude -0.005). Peak input drift (mean PSI) = 9.298 (notable). A sliding-retrained model beat the frozen one (accuracy 0.994 vs 0.994, gap 0.000 but not statistically significant). Champion vs linear baseline: not significant (p=0.226562).

Statistical significance

A score difference is only meaningful if it survives a significance test. Paired predictions on the same held-out rows are compared with the McNemar's test. A p-value below 0.05 means the difference is unlikely to be chance; above it, the two models are statistically indistinguishable - which is itself worth reporting.

comparison	better model	p-value	conclusion
champion vs runner up	tie	1	not significant
champion vs linear baseline	random_forest_bayes	0.2266	not significant
static vs adaptive	adaptive	1	not significant

champion vs runner up: random_forest_bayes and random_forest_random made identical predictions on every test row.

champion vs linear baseline: random_forest_bayes and logreg_none disagree on 11 rows, split fairly evenly. p=0.2266 >= 0.05, so the difference is NOT statistically significant.

static vs adaptive: static and adaptive disagree on 1 row, split fairly evenly. p=1 >= 0.05, so the difference is NOT statistically significant.

APPENDIX A

Scope and limitations

Tuning is measured, not assumed

The point of Stage 3 is to quantify the cost/benefit of tuning, not to prove that tuning always wins. On well-conditioned datasets a regularized baseline can match a tuned booster, and this report says so when it happens.

Drift is detected, not explained

PSI and prequential error are cheap, reliable indicators that a distribution has moved. Neither attributes a cause; deeper causal drift analysis is out of scope.

Single-machine scale

The pipeline targets datasets that fit in memory on one machine. It is not a distributed training system.

One split, one seed

Results come from a single temporal split at a fixed random seed. That is the right choice for a time-ordered problem - shuffled k-fold would leak the future - but it means confidence intervals across resamples are not reported, which is why the significance tests above matter.

APPENDIX B

Glossary

ROC-AUC	Probability a random positive is ranked above a random negative. 0.5 is chance, 1.0 is perfect. Insensitive to class imbalance.
RMSE / MAE	Average prediction error in the target's own units. Lower is better; RMSE punishes large misses harder than MAE.
R-squared	Share of the target's variance the model explains. 0 means no better than predicting the mean.
Regularization (L1/L2)	A penalty on coefficient size that trades a little bias for less variance. L1 can zero coefficients outright, performing feature selection; L2 shrinks them smoothly.
Stacking	Training a small meta-model on the base models' predictions so it can learn which to trust where.
Bayesian optimization	Tuning that builds a surrogate model of the search space to choose each next trial, rather than sampling blindly.
SHAP value	A feature's signed contribution to one prediction, derived from cooperative game theory. Contributions sum to the prediction.
PSI	Population Stability Index: how far a feature's distribution has moved from its training reference. Rules of thumb: below 0.1 stable, 0.1-0.25 moderate, above 0.25 material.
Prequential evaluation	Test-then-train: each chunk is predicted before the model is allowed to learn from it, so no future information leaks.
McNemar's test	A paired test for two classifiers on identical rows, based only on the cases where they disagree.

Produced by the TabML-Guard pipeline.

~ Ayush Gole